Practice 06 — IAM and Least Privilege

Objective: give a user exactly the permissions their job needs and **prove it**: show that what they're allowed to do works and that everything else fails. This is the skill from today's lecture.

Timebox: ~55–60 min, in class. If you've been stuck on one step for more than 10 minutes, stop and ask. Don't push through alone.

> **How you're checked:** this practice is done in class and checked **orally**. There is no report. Keep your terminal open, be ready to **show** each item in the *Definition of done* and **explain** what you did. The instructor will ask a few of the questions under "Be ready to explain".

Using AWS Academy Learner Lab

Our AWS access is the **AWS Academy Learner Lab**. Per the Learner Lab Readme, it does **not** let you create IAM users, groups, custom policies or roles. You only get the ready-made `LabRole` and `LabInstanceProfile`, and the IAM Policy Simulator isn't available. So **everyone does the IAM tasks on moto**, a local AWS emulator that really checks permissions. The commands are the same IAM API calls you'd run on a real account.

Learner Lab is only used for the short, optional **Track A — look around** part at the end.

	In class (everyone)	**Optional, if time is left**
Where	**moto** on your laptop, with permission checks turned on	AWS Academy Learner Lab
Needs	Python 3.9+, AWS CLI, ~5 min setup	Learner Lab started (green dot)
Tasks	1, 2, 3, 4a	Track A — look around

How moto maps to a real AWS account:

Real AWS	**moto equivalent**
AWS account	Local moto server (`http://localhost:5002` , account `123456789012`)
Root user	The 3 "bootstrap" calls moto allows before it starts checking permissions
IAM users, groups, policies	The same IAM API calls, enforced by moto
Access keys / CLI profiles	Real `aws configure --profile` profiles, pointed at moto
Policy Simulator	You predict the result first, then run the request and compare
CloudTrail	moto's terminal log of every request

Definition of done (things you can show)

□ An `admins` group with admin permissions, and your admin identity is a member. No daily work is done with the bootstrap ("root") key.
□ A `developers` group with a customer-managed policy `dev-s3-prefix` that allows reading and writing **only** `course-data/dev/*`.
□ A user `dana` in `developers`, with a CLI profile `dev`.
□ The Task 3 table filled in for rows 1–6: your prediction, the actual result, and which rule decided it.
□ The explicit-deny test: `dana` is denied `dev/secrets/*` even though `dev/*` is allowed.

Task 1 — Set up moto

1. Install the tools. With uv:

```
uv tool install "moto[server]"
uv tool install awscli
```

Or use plain `pip install "moto[server]" awscli`. On Windows, run all of this inside WSL.

2. Start moto in a **separate terminal** and leave it running. `INITIAL_NO_AUTH_ACTION_COUNT=3` means the first 3 requests are free (your "root" moment) and every request after that is checked against IAM policies:

```
INITIAL_NO_AUTH_ACTION_COUNT=3 moto_server -p 5002
```

3. In your working terminal, point the CLI at moto and use the 3 free calls to create an admin. **Do exactly these three commands, in order.** Any extra request uses up a free call:

```
export AWS_ENDPOINT_URL=http://localhost:5002
export AWS_DEFAULT_REGION=us-east-1
export AWS_ACCESS_KEY_ID=bootstrap AWS_SECRET_ACCESS_KEY=bootstrap

cat > admin.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":"*","Resource":"*"}]}
EOF

aws iam create-user --user-name admin                                    # free call 1
aws iam put-user-policy --user-name admin --policy-name admin-all \
    --policy-document file://admin.json                                   # free call 2
aws iam create-access-key --user-name admin                               # free call 3 — copy the
keys!
```

4. Save the admin keys as a profile and stop using the bootstrap key:

```
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
aws configure --profile admin            # paste AccessKeyId + SecretAccessKey, region us-east-1
aws sts get-caller-identity --profile admin
aws iam list-users --profile admin       # works; the bootstrap key is now rejected
```

5. Create the admins group properly and the bucket:

```
aws iam create-group --group-name admins --profile admin
aws iam put-group-policy --group-name admins --policy-name admin-all \
    --policy-document file://admin.json --profile admin
aws iam add-user-to-group --group-name admins --user-name admin --profile admin

export BUCKET=course-data
aws s3 mb s3://$BUCKET --profile admin
echo "seed" > seed.txt && echo "hello" > hello.txt
aws s3 cp seed.txt s3://$BUCKET/prod/seed.txt --profile admin
```

> Keep `AWS_ENDPOINT_URL` set in every terminal you use, otherwise the CLI talks to real AWS.

Task 2 — A least-privilege developer

Save as `dev-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOnlyDevPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::course-data",
      "Condition": { "StringLike": { "s3:prefix": ["dev/*"] } }
    },
    {
      "Sid": "ReadWriteDevObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::course-data/dev/*"
    }
  ]
}
```

```
ACCOUNT=$(aws sts get-caller-identity --profile admin --query Account --output text)

aws iam create-group --group-name developers --profile admin
aws iam create-policy --policy-name dev-s3-prefix \
    --policy-document file://dev-policy.json --profile admin
aws iam attach-group-policy --group-name developers \
    --policy-arn arn:aws:iam::$ACCOUNT:policy/dev-s3-prefix --profile admin

aws iam create-user --user-name dana --profile admin
aws iam add-user-to-group --group-name developers --user-name dana --profile admin
aws iam create-access-key --user-name dana --profile admin    # copy the keys
aws configure --profile dev                                   # paste dana's keys
aws sts get-caller-identity --profile dev                     # → .../user/dana
```

The policy is attached to the **group**, not the user. Be ready to explain why.

Task 3 — Prove it: allowed vs denied

Before running each command, write down what you expect: ✅ or ❌. Then run it as `dana` (`--profile dev`) and fill in the table:

#	Request	Predicted	Actual	Decided by
1	`aws s3 cp hello.txt s3://$BUCKET/dev/hello.txt`			
2	`aws s3 cp s3://$BUCKET/dev/hello.txt -`			
3	`aws s3 cp hello.txt s3://$BUCKET/prod/hello.txt`			
4	`aws s3 cp s3://$BUCKET/prod/seed.txt -`			
5	`aws iam list-users`			
6	`aws ec2 describe-instances`			

"Decided by" is one of: *Allow statement* `<Sid>`, *explicit deny*, or *implicit deny (no matching allow)*.

> **moto note:** moto doesn't support S3 **listing** (`aws s3 ls`) while permission checks are on. It fails with `SignatureDoesNotMatch`, which is a moto limitation, not your mistake. That's why the table has no listing rows.

Task 4a — Explicit deny beats allow

Save as `deny-secrets.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "s3:*",
    "Resource": "arn:aws:s3:::course-data/dev/secrets/*"
  }]
}
```

```
aws iam put-user-policy --user-name dana --policy-name deny-secrets \
    --policy-document file://deny-secrets.json --profile admin

aws s3 cp hello.txt s3://$BUCKET/dev/secrets/db.env --profile dev   # predict first
aws s3 cp hello.txt s3://$BUCKET/dev/ok.txt --profile dev           # predict first
```

Cleanup

Press `Ctrl+C` in the moto terminal. Everything lives in memory and disappears. Remove the `admin` and `dev` profiles from `~/.aws/` so they don't get confused with real profiles later.

Track A — look around in Learner Lab (optional, if time is left)

Start the Learner Lab (**Start Lab**, wait for the green dot, then **AWS**). You can't create IAM users there, but you can see how a real account is wired:

1. In the console, open **IAM** → **Roles** and find `LabRole`. Look at its **Trust relationships** tab: who is allowed to assume it?
2. Open the Learner Lab **AWS Details** panel, copy the CLI credentials into `~/.aws/credentials` (as a `[lab]` profile), and run:

```
aws sts get-caller-identity --profile lab
```

The `Arn` shows an **assumed role** (`assumed-role/voclabs/...`), not an IAM user. The key ID starts with `ASIA` : temporary credentials.

3. **Stretch:** launch a `t3.micro` Amazon Linux instance with **IAM instance profile = `LabInstanceProfile`** , connect with EC2 Instance Connect, and run `aws sts get-caller-identity` on it without configuring any keys. Terminate the instance when you're done.

Homework (not in class)

These two tasks are part of the topic but don't fit in the hour. Do them at home on moto; the instructor may ask about them at the next oral check.

H1 — Temporary credentials through a role (was Task 4b)

```
cat > trust.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Principal":{"AWS":"arn:aws:iam::$ACCOUNT:user/dana"},"Action":"sts:AssumeRole"}]}
EOF
cat > reader.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Action":"s3:GetObject","Resource":"arn:aws:s3:::$BUCKET/prod/*"}]}
EOF

aws iam create-role --role-name prod-reader \
    --assume-role-policy-document file://trust.json --profile admin
aws iam put-role-policy --role-name prod-reader --policy-name read-prod \
    --policy-document file://reader.json --profile admin

# Try it — predict first:
aws sts assume-role --role-arn arn:aws:iam::$ACCOUNT:role/prod-reader \
    --role-session-name test --profile dev
```

On moto that fails until dana's **own** permissions also include `sts:AssumeRole`:

```
cat > allow-assume.json <<EOF
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
 "Action":"sts:AssumeRole","Resource":"arn:aws:iam::$ACCOUNT:role/prod-reader"}]}
EOF
aws iam put-user-policy --user-name dana --policy-name allow-assume \
    --policy-document file://allow-assume.json --profile admin

aws configure set role_arn arn:aws:iam::$ACCOUNT:role/prod-reader --profile prod-reader
aws configure set source_profile dev --profile prod-reader
aws configure set region us-east-1 --profile prod-reader

aws sts get-caller-identity --profile prod-reader              # → assumed-role/prod-reader/...
aws s3 cp s3://$BUCKET/prod/seed.txt - --profile prod-reader   # works
aws s3 cp hello.txt s3://$BUCKET/prod/x --profile prod-reader  # AccessDenied
```

Run `aws sts assume-role ...` once by hand and find the `ASIA…` key ID, the `SessionToken` and the `Expiration` in the output.

H2 — Policy debugging (was Task 5)

Each policy below was written by a teammate. For each one, say **what is wrong**, **what actually happens**, and give a **fixed** version. You can check your fix on moto by attaching it to a new test user.

H2.1 — "Dana gets AccessDenied reading `reports/q1.csv`, but the policy clearly allows GetObject!"

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::course-data" }] }
```

H2.2 — "The CI bot only needs to upload build artifacts."

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": "s3:*",
    "Resource": "*" }] }
```

H2.3 — "We denied deletes, but the intern still deleted a file."

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "s3:*",
      "Resource": "arn:aws:s3:::course-data/*" },
    { "Effect": "Deny", "Action": "s3:DeleteBucket",
      "Resource": "arn:aws:s3:::course-data/*" } ] }
```

H2.4 — "Listing `dev/` works, but downloading anything from it fails."

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Action": ["s3:ListBucket", "s3:GetObject"],
    "Resource": "arn:aws:s3:::course-data",
    "Condition": { "StringLike": { "s3:prefix": ["dev/*"] } } }] }
```

Be ready to explain

1. Why is the developer policy attached to a group instead of the user?
2. Rows 5 and 6 in Task 3 were denied, yet no policy mentions IAM or EC2. Why?
3. In Task 4a, both an allow and a deny matched the same request. Which won, and what is the general rule?
4. Why does `dev-policy.json` need two statements, one for `ListBucket` and one for `GetObject` / `PutObject`?

5. Why can't we create IAM users in Learner Lab, and what does `get-caller-identity` show there instead?

Common mistakes

- **`GetObject` denied, but the policy "allows it".** Check the resource: object actions need `bucket/*` or `bucket/prefix/*`; bucket actions like `ListBucket` need the bare bucket ARN.
- **Everything returns `InvalidClientTokenId`.** You used more than 3 requests before saving the admin key, or you're still exporting `AWS_ACCESS_KEY_ID=bootstrap`. Restart `moto_server` (it forgets everything) and redo Task 1 exactly.
- **Commands suddenly hit real AWS.** You opened a new terminal and forgot `export AWS_ENDPOINT_URL=http://localhost:5002`.
- **Trying to create an IAM user in Learner Lab.** It's blocked there; that's expected. IAM tasks are done on moto.
- **Keys on screen.** Don't show or share a `SecretAccessKey` or `SessionToken`, even the moto ones. Get used to hiding them.